

SudoOS-Plus

中文设计方案与进展报告

OSCOMP 2025 参赛项目 · 正式提交版 v1.0

双架构支持 · 内核与用户态闭环 · 评分基线 · 文件级实现说明 · 合规声明

当前提交：2030f7f

当前总分：741.9279616944203

更新日期：2026-06-30

目录

SudoOS-Plus : 双架构 Rust 操作系统设计方案与 OSCOMP 进展报告	5
1. 项目摘要	5
1.1 正式交付物	5
1.2 项目来源与参考边界	5
1.3 基线对比口径	6
1.4 参考版本与增量贡献	6
2. 当前结论	7
2.1 已经形成的能力	7
2.2 当前最重要的事实边界	7
3. 当前平台得分	8
3.1 截图原始数据	8
3.2 得分结构	8
3.3 得分解读	8
4. 设计目标与边界	9
4.1 设计目标	9
4.2 当前非目标或未完成目标	9
5. 总体架构	9
6. 启动与初始化流程	10
6.1 RISC-V 64	10
6.2 LoongArch 64	10
6.3 公共 kernel_main 顺序	10
7. 各子系统详细实现	11
7.1 架构抽象与启动协议	11
7.2 物理内存与内核堆	11
7.3 页表、VMA 与用户地址空间	11
7.4 中断、时间与并发	11
7.5 SMP、IPI、调度与 TLB	12
7.6 进程、线程与 futex	12
7.7 ELF、动态装载与用户栈	12
7.8 系统调用兼容	13
7.9 VFS、文件系统与终端	13
7.10 设备、VirtIO 与块层	14
7.11 网络、随机数与 RTC	14
7.12 OSCOMP 评分执行器	14
8. 当前进度矩阵	15
9. 本轮可复现实测	15

9.1 myos-mm 单元测试	15
9.2 提交总审计	16
9.3 newtest 静态里程碑	16
9.4 baseline guard	16
10. 当前构建产物	17
11. 构建、运行与验证	17
11.1 比赛构建	17
11.2 开发构建与 QEMU	17
11.3 常用验证	17
11.4 本地 contest	17
12. 文件与目录功能说明	18
12.1 根目录	18
12.2 kernel/src	19
12.3 mm/src	21
12.4 arch/riscv64	22
12.5 arch/loongarch64	23
12.6 基础 crate	23
12.7 scripts/	23
12.8 docs/	25
13. 当前问题与风险	26
14. 更新后的实施方案	26
阶段 A：恢复可信门禁 (P0)	26
阶段 B：冻结并复现当前 741.9279 基线 (P0)	26
阶段 C：稳定 LoongArch busybox (P0)	26
阶段 D：修复 P2/P4 契约 (P1)	27
阶段 E：提高 libcbench (P1)	27
阶段 F：形成 lmbench 得分 (P1)	27
阶段 G：ext4 与 iozone (P2)	27
阶段 H：网络评分 (P2)	27
阶段 I：兼容性扩展 (P3)	27
15. 里程碑与完成定义	28
15.1 已完成或基本完成	28
15.2 当前阶段完成定义	28
16. 维护与提交约束	28
17. 最终判断	29
附录 A：生成式人工智能使用声明	29
附录 B：第三方代码与依赖声明	29
B.1 直接 vendored 项目	29
B.2 Cargo 离线依赖	30

B.3 许可证与再分发说明	30
-------------------------	----

SudoOS-Plus：双架构 Rust 操作系统设计方案与 OSCOMP 进展报告

本文档面向项目设计评审、阶段汇报、比赛提交与后续开发交接。文档版本：正式提交版 v1.0

状态日期：2026-06-30 (Asia/Shanghai) 当前分支：[6.28](#)

当前提交：[2030f7ff71345ba821d49bb0f743267deea809f4](#) (2030f7f) 工具链：[nightly-2025-01-18](#)，Rust 2024 Edition 评分来源：2026-06-28 平台截图 评分总计：741.9279616944203

1. 项目摘要

SudoOS-Plus 是一个以 Rust `no_std` 为主体、同时支持 RISC-V 64 与 LoongArch 64 的教学/比赛型操作系统内核。项目已从最初的双架构高半内核启动，逐步扩展到内存管理、SMP、抢占调度、进程与线程、Linux `asm-generic` 64 位系统调用、ELF 装载、VFS、`ext4` 只读访问、VirtIO 设备、基础网络接口与 OSCOMP `sdcard` 测试调度。

当前平台截图表明，四个架构/LibC 组合已经在 `basic`、`busybox`、`libcbench` 和 `lua` 四组形成有效得分，合计 741.9279616944203。另一方面，`cyclictest`、`iozone`、`iperf`、`libctest`、`lmbench`、`ltp` 和 `netperf` 仍为 0 分或未计分，说明项目已经拥有可运行的 Linux-like 基础，但距离完整性能、网络、文件系统与兼容性覆盖仍有明显空间。

本文严格区分三类事实：

- 源码存在：当前提交中可以定位到实现；
- 本地验证：本轮实际执行的静态审计、单元测试或现有构建产物证据；
- 平台评分：仅以附件截图为依据，不用源码推测平台结果。

“实现存在”不自动等同于“平台已通过”；“历史曾通过”也不自动等同于“当前 HEAD 已回归通过”。

1.1 正式交付物

交付物	文件名	用途
源码与在线设计说明	README.md	仓库首页、设计审查与开发交接
设计方案 PDF	SudoOS-Plus-设计方案与进展报告.pdf	正式书面材料
进展汇报 PPT	SudoOS-Plus-设计方案与进展汇报.pptx	答辩与阶段汇报
演示视频	百度网盘链接 ，提取码： 77FB	系统运行与项目成果演示
AI 使用声明	AI-使用声明.md	四份材料统一声明的独立文本源

PDF 由本 README 的正式提交版直接生成，因此章节、数据、AI 声明和第三方代码声明与 README 保持一致。PPT 使用相同事实口径，并在末尾单列 AI 使用声明和附录 B 摘要。

演示视频备用说明：复制链接和提取码后，可使用浏览器或百度网盘 APP 打开。

1.2 项目来源与参考边界

本仓库现有 Git 历史中可追溯的最早内部基线为：

项目	内容
内部参考版本	453eb91 (competition: final-test submission)
基线日期	2026-06-22
当前比较版本	2030f7f

项目	内容
外部竞赛 OS 源码基线	未发现；项目不是从其他参赛队仓库整体派生
第三方代码	仅限 vendor/ 和 Cargo 依赖，详见附录 B

架构和 ABI 设计会参考公开规范及 Linux 行为，但“参考规范”不等于复制 Linux 内核源码。项目自研代码与第三方源码通过目录边界、Cargo 依赖和许可证文件区分。

1.3 基线对比口径

为避免把离线依赖体积误写成团队代码贡献，增量统计排除 vendor/** 和二进制 PPT：

```
453eb91..2030f7f
首方文件：117 个
新增：18,084 行
删除：3,072 行
新增文件：67 个
修改文件：50 个
```

完整仓库 diff 会包含离线 Cargo、Rust 源码和工具链快照，规模远大于首方统计；这些文件的引入属于可复现构建工作，不作为原创内核代码行数。

1.4 参考版本与增量贡献

相对于内部参考版本 453eb91，当前版本的主要增量贡献如下：

- 双架构比赛构建链：重构根 Makefile、保留 Makefile.project，加入离线 Cargo、rust-src、双内核产物和提交审计；
- RISC-V 启动稳定性：修复链接段对齐、高半栈切换、早期 trap、正式页表和 buddy handoff；
- LoongArch 兼容性：补充 FPU/FPD、用户异常、动态 loader、BusyBox 与 glibc/musl 组合路径；
- Linux-like ABI：扩展文件、进程、线程、signal、时间、调度、futex、socket 和资源限制 syscall；
- 用户态运行链：完善 ELF/ET_DYN/PT_INTERP、auxv、TLS、clone/futex 与当前镜像替换；
- VFS 与设备：增加 procfs、sysfs、devpts、RTC、RNG、设备模型、VirtIO block/net 和 ext4 只读访问；
- 评分执行器：实现 sdcard 有界发现、架构/LibC 分组、预算、真实退出码、summary、score 与统一关机；
- 验证工程：新增 P0-P6、M5-M16、启动/页分配/动态 ELF/网络等静态审计和 smoke/stress/preflight；
- 文档与交付：形成当前 README、正式 PDF、汇报 PPT、AI 声明与第三方代码附录。

团队成员通过 Git 作者信息保留人的贡献归属；生成式 AI 仅作为辅助工具，不作为作者或共同作者。具体声明见附录 A。

2. 当前结论

2.1 已经形成的能力

- 双架构裸机启动、高半内核、FDT 解析和统一 `BootInfo` ；
- RISC-V Sv39 与 LoongArch DMW + 四级页表 ；
- buddy、slab、large allocation、vmalloc/ioremap 与用户地址空间 ；
- 双架构 trap/IRQ、timer、SMP、IPI、TLB shutdown、抢占式调度 ；
- Process/Thread 强所有权、fd table、signal state、进程组与 session ；
- Linux asm-generic 64 位 syscall 编号与大规模兼容实现 ；
- ELF64、`PT_LOAD`、`PT_INTERP`、auxv、初始用户栈和 `execve` ；
- tmpfs/devfs/procfs/sysfs、pipe、TTY、PTY、mount table ；
- VirtIO block、buffer/page cache、ext4 只读目录和文件路径访问 ；
- VirtIO-Net/smoltcp 接口骨架与 socket syscall ；
- OSCOMP sdcard 扫描、测试分组、预算、summary、score 与关机路径。

2.2 当前最重要的事实边界

- 附件总分是 741.9279616944203，但仓库中没有与截图一一对应的完整平台日志 ；
- 当前 `kernel-rv`、`kernel-la` 已存在，并与现有专项文档记录的 hash 一致 ；
- `cargo test -p myos-mm` 本轮为 45 passed / 0 failed，现有 README 中 “24 tests” 的旧口径已经过时 ；
- `oscomp-audit.py` 本轮为 11 PASS / 2 WARN / 0 FAIL ；
- newtest 静态审计中 P0、P3、P5、P6 通过，P2、P4 未全部通过 ；
- `oscomp_baseline_guard.py` 输出 240 PASS / 9 WARN / 0 FAIL，但其 `check()` 函数没有使用 `ok` 参数，当前输出不能作为可信质量门禁 ；
- 当前提交是将 P14L/P14M/P14N 整体回退到 P14K 后的基线，仍需同一 HEAD 下的双架构完整 contest 日志闭环。

3. 当前平台得分

3.1 截图原始数据

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	97	97	97	97	388
busybox	53	54	53	54	214
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	0.0	0.0	0.0	0.0	0.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	27.143920815559106	28.37984327461565	22.936107443964225	25.468090160281417	103.9279616944203
libctest	-	-	0	0	0
lmbench	0.0	0.0	0.0	0.0	0.0
ltp	0	0	0	0	0
lua	9	9	9	9	36
netperf	0.0	0.0	0.0	0.0	0.0
总分	186.1439208155591	188.37984327461564	181.93610744396423	185.4680901602814	741.9279616944203

3.2 得分结构

维度	得分	占总分
basic	388	52.30%
busybox	214	28.84%
libcbench	103.9279616944203	14.01%
lua	36	4.85%
其余七组	0	0%

架构与 LibC 汇总：

汇总维度	得分
RISC-V (glibc + musl)	373.8479334348971
LoongArch (glibc + musl)	368.0800282595234
glibc (LA + RV)	374.5237640901747
musl (LA + RV)	367.4041976042456

3.3 得分解读

- 四个组合的 **basic** 都是 97，说明基础 syscall/进程/VFS 路径覆盖较均衡；
- RISC-V 的 **busybox** 比 LoongArch 每个 LibC 多 1 分，LoongArch 仍有 applet、shell 或动态装载边界；
- libcbench** 是四组合差异最大的组，musl-la 最低，表明 libc 线程、同步、调度或时间 ABI 仍是重点；
- lua** 四组合均为 9，动态 ELF、基础文件访问和用户态运行链已具备可见成果；

- 网络测试为 0 不能简单解释为“没有网络源码”：当前确有 socket/VirtIO-Net/smoltcp 代码，但评分开关关闭、真实数据面与测试闭环尚未完成；
- **Imbench** 截图为 0，与仓库中“RV glibc 六项 mini runner”的源码并不矛盾：runner 存在不代表平台评分器已采纳当前输出。

4. 设计目标与边界

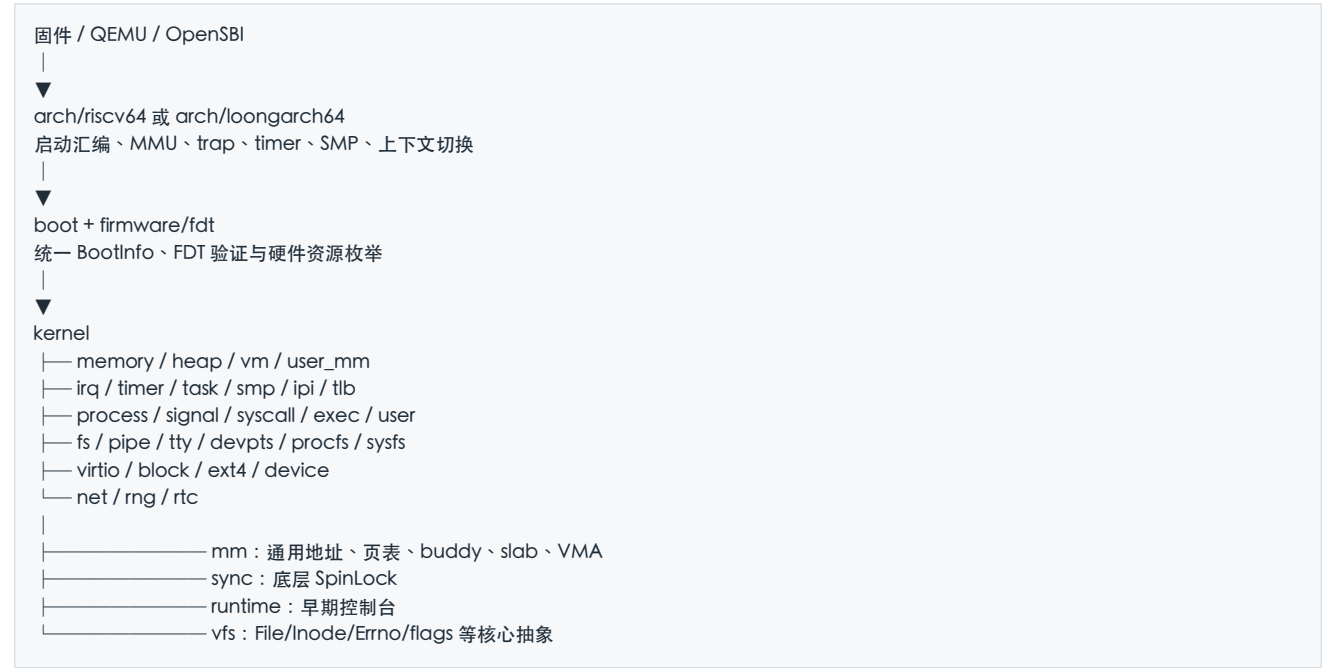
4.1 设计目标

- 双架构共享尽可能多的内核策略，只把硬件机制留在 **arch/***；
- 采用 Linux asm-generic 64 位 ABI，降低 glibc/musl/BusyBox 移植成本；
- 让内存、调度、进程、VFS、驱动形成可组合的真实闭环，而不是测试桩；
- 所有评分 PASS 可追溯到真实二进制、参数、退出码和解析结果；
- 通过离线 vendor、固定工具链和 **make all** 满足比赛可复现构建。

4.2 当前非目标或未完成目标

- 完整 POSIX/Linux signal 语义与 syscall restart；
- 完整写入型 ext4、日志、崩溃一致性与持久化根文件系统；
- 完整 TCP/IP 稳定性、DNS、TLS 与真实网络评分；
- 完整 LTP、libctest、iozone、cyclictest、netperf；
- 高性能 COW fork、页缓存统一、NUMA/IOMMU 等高级能力；
- 实体机驱动覆盖；当前主要验证环境仍是 QEMU。

5. 总体架构



Cargo workspace 当前包含 9 个成员：

Crate	路径	主要职责
myos-kernel	kernel/	内核入口与全部系统服务集成

Crate	路径	主要职责
arch-riscv64	arch/riscv64/	RISC-V 启动、Sv39、SBI、trap、SMP
arch-loongarch64	arch/loongarch64/	LoongArch DMW/页表、trap、SMP、QEMU virt
myos-boot	boot/	跨架构启动参数模型
myos-fdt	firmware/fdt/	FDT blob 验证、解析与资源枚举
myos-mm	mm/	通用内存管理策略与数据结构
myos-runtime	runtime/	字节控制台和格式化适配
myos-sync	sync/	最底层无依赖自旋锁
myos-vfs	vfs/	VFS 公共类型、FileOperations 与 fd 语义

第三方依赖包括本地 `vendor/virtio-drivers`、`vendor/fdt-reader` 和离线 `vendor/cargo`。网络协议依赖 `smoltcp 0.11`，启用了 Ethernet、IPv4、IPv6、TCP、UDP 与 raw socket 特性。

6. 启动与初始化流程

6.1 RISC-V 64

```

OpenSBI
→ 0x80200000 低地址 _start
→ 构造临时 Sv39 页表
→ 写 satp / sfence.vma
→ 跳转 0xfffffff800000000 高半内核
→ 设置 gp、sp、清 BSS
→ rust_entry
→ kernel_main

```

6.2 LoongArch 64

```

QEMU direct boot
→ 0x2000000 _start
→ 配置 DMW0 (uncached) / DMW1 (cached)
→ CRMD.PG 开启
→ 切换 cached 高地址执行
→ 解析 EFI-style 参数与 FDT
→ rust_entry
→ kernel_main

```

6.3 公共 `kernel_main` 顺序

当前源码中的关键顺序如下：

- 读取并验证 FDT；
- 枚举内存、CPU、VirtIO MMIO、PCI host 和 initrd；
- 构造物理内存排除表；
- 验证分页策略和 early frame allocator；
- 映射 FDT、内核镜像、direct map 与 SMP trampoline；
- 安装最终页表，结束 early allocator 生命周期；
- 初始化 buddy page allocator 和全局 heap；
- 初始化 trap、IRQ、clock、timer、vmalloc；
- 初始化 VirtIO、设备模型、RNG、网络、RTC、fault；
- 初始化 VFS，挂载 `/proc`、`/sys`，解包 initramfs；

- 发现并挂载 `/dev/vda`，建立 `/mnt/sdcard` 测试环境；
- 初始化 TTY、任务系统、SMP secondary CPU 与 workqueue；
- 运行用户态/BusyBox/sdcard verifier；
- 输出 `SMOKE_TEST: PASS` 或 `OSCOMP summary/score`；
- 比赛模式统一关机，普通 smoke 进入 idle。

初始化顺序的核心约束是：页分配器先于 heap，trap/IRQ 在中断打开前完成，任务系统 先于 secondary CPU 正式参与调度，VFS 与动态装载依赖的设备/内存子系统必须先就绪。

7. 各子系统详细实现

7.1 架构抽象与启动协议

- `rust_entry(arg0, arg1, arg2)` 是双架构公共 Rust 入口；
- 每个架构把固件参数转换为 `BootContext`，再生成公共 `BootInfo`；
- 架构 crate 负责 CSR/寄存器、页表格式、trap frame、timer 与上下文切换；
- 内核策略代码通过 `crate::arch` 选择当前架构；
- 非 RISC-V/LoongArch 架构会在编译期失败，避免静默构建错误目标。

7.2 物理内存与内核堆

启动期先使用固定容量 `MemoryMap` 和 `EarlyFrameAllocator`，从 FDT RAM 中排除固件、内核镜像、FDT、initrd、启动栈和架构保留区。正式阶段把剩余内存交给 buddy，按 DMA32/Normal zone 管理，并维护页引用计数。

堆分为两条路径：

- 小对象：9 个 size class (8 B 到 2048 B) 的 slab cache；
- 大对象：向 page provider 申请连续页，并在 allocation header 中记录元数据。

`KernelVirtualAllocator` 为 `vmalloc/ioremap` 预留带 guard page 的虚拟区间；`RuntimePageTable` 负责运行期 map/protect/unmap/translate。释放 API 消费 token，降低 double free/unmap 风险。

7.3 页表、VMA 与用户地址空间

- RISC-V：3 级 Sv39、512 项页表、39 位虚拟地址；
- LoongArch：4 级页表、DMW 直映、TLB refill 与硬件页表 CSR；
- 通用 `MappingOptions` 强制 `W^X`，拒绝不安全的 writable + executable；
- `VmAreaSet` 提供有序插入、gap 查找、split/coalesce；
- `AddressSpace` 管理 brk、mmap、munmap、mprotect；
- `UserMm` 管理独立根页表、ASID generation、active CPU mask 与 per-mm TLB；
- fault pipeline 区分 anonymous、file、device、COW、protection 与 segv；
- 用户拷贝失败按 `-EFAULT` 返回，内核 fault 默认 fail-fast。

7.4 中断、时间与并发

- `trap` 安装双架构入口并验证 frame guard；
- `irq` 把 timer/software/external/platform 中断统一分类；
- `time` 提供 monotonic tick 与 clockevent；

- `timer` 提供 one-shot、取消、timeout 与队列；
- `workqueue` 支持普通与 delayed work；
- `IrqSpinLock` 保存/恢复本地中断状态；
- `lockdep` 记录 LockClass/LockRank 和当前 CPU 持锁链；
- `tracked_spin` 增加 owner、迁移固定与 lockdep 跟踪。

7.5 SMP、IPI、调度与 TLB

双架构最多按 `MAX_CPUS=8` 管理 CPU。CPU 生命周期区分 `discovered`、`online`、`active` 与 `IPI-ready`。secondary CPU 使用独立启动栈、trap 状态和 timer。

调度器当前实现：

- per-CPU FIFO round-robin；
- idle task；
- 抢占与 `sched_yield`；
- WaitQueue/Completion；
- work stealing 与任务迁移；
- task reaper 与 deferred 资源回收；
- 64 KiB guarded kernel stack。

IPI mailbox 支持 `reschedule`、TLB shutdown 和 `call-function`。TLB request 使用 request ID、target mask、completion mask，支持 page/range 与长范围 fallback。

7.6 进程、线程与 futex

`Process` 拥有：

- `UserMm`；
- `FileTable`；
- `SignalState`；
- `Credentials`；
- `root/cwd`；
- `child/zombie` 状态；
- `process group/session`。

`Thread` 拥有用户 trap frame、TLS、`clear-child-tid`、robust list、signal mask 和 scheduler task。`clone/clone3` 支持 fork-like 进程和 `CLONE_VM` 线程分支，处理 `CLONE_SETTLS`、`CLONE_CHILD_CLEARTID`；futex key 包含 mm ASID，避免不同地址空间同虚拟地址相互干扰。

7.7 ELF、动态装载与用户栈

`elf.rs` 负责 ELF64 header/program header 校验与 metadata；`exec.rs` 负责真正构造进程镜像：

- 映射 `PT_LOAD`，按 ELF flags 设置权限；
- 处理 BSS 清零；
- 支持静态 ELF 与 `ET_DYN`；
- 支持 `PT_INTERP` 动态解释器交接；
- 构造 `argc/argv/envp/auxv`；

- 提供 `AT_PAGESZ`、`AT_ENTRY`、`AT_PHDR`、`AT_PHNUM`、`AT_RANDOM`、

`UID/GID`、`HWCAP`、`PLATFORM` 等；

- `execve` 替换当前 `mm`，执行 `CLOEXEC`，销毁旧地址空间。

`sdcard` 启动路径会把 `glibc/musl loader` 和共享库从 `ext4` 物化到 `/lib`、`/lib64` 与 `/usr/lib`。当前 `LoongArch` 的 `loader` 别名与 `P4` 审计规则仍有契约不一致。

7.8 系统调用兼容

`syscall.rs` 当前声明 118 个 `asm-generic` 编号常量；`user.rs` 负责 `dispatch` 和 具体策略。主要覆盖：

类别	代表 syscall
文件	<code>openat/close/read/write/readv/writev/pread64/pwrite64/lseek/fstat/statx</code>
目录/VFS	<code>getcwd/chdir/getdents64/mkdirat/unlinkat/renameat/renameat2/linkat/symlinkat</code>
fd	<code>dup/dup3/fcntl/ioctl/pipe2/fsync/fdatasync/ftruncate</code>
内存	<code>brk/mmap/munmap/mprotect/pkey_mprotect/mlock/munlock</code>
进程	<code>clone/clone3/execve/exit/exit_group/wait4/getpid/getppid/gettid</code>
线程	<code>set_tid_address/set_robust_list/get_robust_list/futex/rseq</code>
signal	<code>kill/tkill/tgkill/rt_sigaction/rt_sigprocmask/rt_sigreturn/altstack</code>
时间	<code>nanosleep/clock_gettime/clock_getres/clock_nanosleep/gettimeofday/setitimer</code>
调度	<code>sched_yield/affinity/scheduler/param/priority/rr_get_interval</code>
系统	<code>uname/sysinfo/prlimit64/getrusage/getrandom/prctl/syslog</code>
网络	<code>socket/bind/listen/accept/connect/sendto/recvfrom/shutdown/socketopt</code>

并非每个兼容 `syscall` 都实现了 `Linux` 的全部 `flag`、竞态和边界语义；表格表示当前 存在可定位实现，不表示 `LTP` 全通过。

7.9 VFS、文件系统与终端

`myos-vfs` 定义 `File`、`FileOperations`、`open flags`、`poll events`、`stat`、`dirent` 和 `errno`。内核 `fs` 层在其上提供：

- `tmpfs` 根；
- `devfs` 与标准设备；
- `initramfs newc` 解包；
- `mount table`；
- `procfs` 与 `sysfs`；
- `symlink/hardlink`；
- `path resolve`、`cwd/root`；
- `ext4 subtree`/单文件物化；
- 标准 `fd 0/1/2`。

`pipe` 支持 `blocking` `read/write`、`EOF`、`EPIPE`、`nonblock` 和 `poll`。TTY 支持 `canonical input`、`echo`、`backspace`、`Ctrl-C`、`termios/winsize` 与前台进程组；`devpts` 提供 `/dev/ptmx` 和 `/dev/pts/<N>`。

ext4 当前重点是只读 superblock、inode、extent/indirect block、目录遍历与文件读取。
尚未形成完整写入、journal 与崩溃一致性闭环。

7.10 设备、VirtIO 与块层

device.rs 提供类似 Linux 的 Bus/Device/Driver 抽象。virtio.rs 从 FDT VirtIO-MMIO 或 PCI host 探测设备，通过 SudoHal 提供 DMA、物理/虚拟地址转换。

块层包括：

- BlockDevice trait 与注册表；
- request queue；
- read_at/write_at；
- bounded buffer cache；
- page cache；
- flush；
- memory block device 自测；
- VirtIO block 包装和 /dev/vda。

7.11 网络、随机数与 RTC

网络目录已经包含：

- NetDevice trait 和接口注册表；
- VirtIO-Net raw 驱动包装；
- AF_INET socket；
- TCP/UDP socket state；
- socket、bind、listen、accept、connect、sendto、recvfrom、shutdown；
- poll、FIONBIO、部分 setsockopt/getsockopt。

但当前 OSCOMP_ENABLE_IPERF_MINI=false、OSCOMP_ENABLE_NETPERF_MINI=false，且平台截图两组均为 0。因此网络应标记为“源码路径存在、静态 P6 审计通过、评分未 闭环”，不能标记为完整完成。

RNG 使用 ChaCha20 DRBG，并可从 VirtIO-RNG 播种；RTC 提供统一读取和 /dev/rtc 框架。P2 审计当前指出 rtc.rs 缺少其期待的 RTC_RD_TIME ioctl 常量。

7.12 OSCOMP 评分执行器

当前流程为：

```
ext4 / sdcard
→ bounded 目录扫描与测试脚本发现
→ arch × libc × group 分类
→ whitelist / heavy-skip / preflight
→ direct runner 或 shell runner
→ 真实退出状态 raw
→ PASS / FAIL / SKIP / timeout / signal
→ OS COMP SUMMARY + score
→ 平台关机
```

当前关键开关：

开关	值	影响
OSCOMP_ENABLE_LMBENCH_MINI	true	仅 RV glibc mini 路径

开关	值	影响
OSCOMP_ENABLE_LIBCBENCH_EXTRA	false	不扩展 libcbench
OSCOMP_ENABLE_CYCLICTEST_MINI	false	cyclictest 关闭
OSCOMP_ENABLE_IPERF_MINI	false	iperf 关闭
OSCOMP_ENABLE_NETPERF_MINI	false	netperf 关闭
OSCOMP_ENABLE_LTP_ALLOWLIST	false	LTP 关闭
RV 总预算	420.000 ms	到期停止并保留关机余量
LA 总预算	240.000 ms	到期停止并保留关机余量
RV glibc lmbench 预算	320.000 ms	六项 mini workload

RV glibc lmbench mini 运行 `lat_syscall` 的 `null/read/write/stat/fstat/open` 六项，捕获真实 `stdout/stderr` 后解析 `microseconds`。解析失败必须 `fail-closed`。

当前 P14K 的 LA busybox direct runner 含 55 个 case，并重新包含后台 `sleep 5 + kill $!`，同时允许 glibc outer-shell fallback；这两条路径均有历史 不稳定证据，是当前最高优先级风险之一。

8. 当前进度矩阵

子系统	源码状态	本轮证据	平台得分关联	判定
双架构构建	已实现	两个内核产物存在；总审计 PASS	四组合均有分	稳定基线
启动/FDT/高半	已实现	构建产物 + 大量审计脚本	basic	稳定基线
内存管理	已实现	45/45 单测通过	basic/libcbench	已验证
SMP/调度	已实现	P3 静态审计通过；历史 smoke 文档	libcbench	已实现，需当前动态回归
进程/线程/futex	已实现	P5 静态审计通过	busybox/libcbench	已实现
syscall ABI	大规模实现	P0 静态审计通过	basic/busybox/lua	已实现
ELF/exec	静态/动态路径存在	P4 审计未全通过	busybox/lua	部分闭环
VFS/tmpfs/devfs	已实现	basic/busybox 有平台分	basic/busybox	已验证一部分
procfs/sysfs	已实现	源码与启动挂载路径	busybox/LTP	未单独评分
TTY/PTY/pipe	已实现基础	BusyBox 得分	busybox	基础闭环
VirtIO block	已实现	总审计 PASS	sdcard/全部组	已形成底座
ext4	只读路径为主	总审计 PASS；P2 有 RTC 缺口	sdcard/全部组	部分闭环
网络	接口与 syscall 存在	P6 静态审计通过	iperf/netperf	评分为 0
signal	基础 delivery/return	syscall 与审计证据	libcbench/LTP	不完整
lmbench mini	源码存在	当前截图 0 分	lmbench	未形成评分闭环
质量门禁	脚本丰富	baseline guard 逻辑失真	提交可靠性	P0 修复项

9. 本轮可复现实测

9.1 myos-mm 单元测试

45 passed 0 failed 0 ignored

覆盖地址空间、ASID、CPU mask、early allocator、fault、MemoryMap、页表几何、W^X、用户栈增长、TLB generation handshake、VMA split/coalesce、buddy refcount 和 vmalloc guard。

当前仍有 6 条 `unused_parens` 编译 warning，不影响测试结果，但应在冻结后清理。

9.2 提交总审计

```
PASS=11
WARN=2
FAIL=0
```

两条 WARN：

- 一 本地 `.cargo` 存在，不能依赖 judge clone 保留隐藏目录；
- 一 原始 `Makefile.project` 含 `smoke/QEMU/stress`，必须确保根 `Makefile all`

只走比赛构建包装。

9.3 newtest 静态里程碑

审计	结果	说明
P0 ABI	PASS	uname、auxv、platform/HWCAP、console lockdep
P2 VFS	FAIL	<code>rtc.rs</code> 未满足 <code>RTC_RD_TIME</code> 常量检查
P3 scheduler	PASS	scheduler/affinity/param/priority ABI
P4 dynamic ELF	FAIL	LA loader 路径与 <code>/lib64</code> → <code>/lib</code> 审计契约不满足
P5 clone/futex/TLS	PASS	CLONE_VM、TLS、clear-child-tid、robust list
P6 network	PASS	socket syscall、poll、FIONBIO、sockopt

P4 的第二项与当前源码设计存在直接冲突：源码明确把 `/lib64` 作为真实目录而不是指向 `/lib` 的 symlink。后续必须选择并统一“运行时真实需求”和“审计契约”，不能只为通过字符串检查破坏动态链接器路径。

9.4 baseline guard

脚本当前打印：

```
PASS: 240
WARN: 9
FAIL: 0
BASELINE GUARD: OK
```

但 `check(level, name, ok, detail)` 直接按调用方 `level` 归类，`ok` 没有参与判断。因此即使 `ok=False`，`check(PASS, ..., False)` 仍会进入 PASS。当前还存在可复现反例：guard 宣称 LA busybox 排除了后台 kill case，而 `kernel/src/user.rs` 实际仍有 `sh -c 'sleep 5' & ./busybox kill $!`。

结论：这 240 个 PASS 只能视为检查清单被执行，不能作为可信 CI gate。

10. 当前构建产物

产物	大小	SHA-256	文件时间
kernel-rv	9,761,048 B	b48ef6588457a4f7655bcbcb65e1fb5aee6518812e8badc75caa1c24a1bdc496	2026-06-28 19:40:41 +0800
kernel-la	8,003,496 B	3e587b092589abce624aa74d624e909d9bbb9724ac7529f2e6aaee f39400c7f0	2026-06-28 19:40:41 +0800

这只能证明对应二进制存在且可被审计识别，不等同于当前 HEAD 已完成一轮完整 contest。

11. 构建、运行与验证

11.1 比赛构建

```
make all
```

根 `Makefile` 的 `all` 只调用 `scripts/oscomp-build.sh`，目标是生成 `kernel-rv` 和 `kernel-la`，不应触发 QEMU、smoke、stress 或网络操作。

11.2 开发构建与 QEMU

```
make build ARCH=riscv64
make build ARCH=loongarch64
make run ARCH=riscv64
make run ARCH=loongarch64
```

普通开发目标会转发到 `Makefile.project`。

11.3 常用验证

```
cargo test -p myos-mm
make oscomp-audit
make oscomp-baseline-check
make oscomp-newtest-full-audit
make check
make verify
```

注意：在 `baseline guard` 修复前，`make oscomp-baseline-check` 返回 0 不能证明所有 条件为真；`make oscomp-newtest-full-audit` 当前会因 P2/P4 失败而停止。

11.4 本地 contest

需要 `sdcard-rv.img` / `sdcard-la.img`：

```
make contest-rv
make contest-la
```

推荐把完整串口日志保存到独立目录，并至少扫描：

```
OS COMP SUMMARY
score=
panic
kernel page fault
known-bad
SIGSEGV
signal=11
signal=14
timeout
lmbench-mini: summary
oscomp-la-busybox-direct: summary
```

12. 文件与目录功能说明

12.1 根目录

文件/目录	功能
Cargo.toml	workspace、统一 edition/lint/profile
Cargo.lock	锁定依赖版本
rust-toolchain.toml	固定 nightly
Makefile	比赛提交入口与 OSCOMP 专项审计
Makefile.project	原始开发构建、QEMU、smoke、stress、里程碑门禁
README.md	当前中文设计方案与进展报告
README-OSCOMP-6.28.md	6.28 评分路径专项风险审计
LAST_UPDATE.md / update.md	历史更新与交接记录
newtest_fullscore_gap_plan.md	newtest 满分缺口规划
.cargo/config.toml	当前 Cargo 离线源与 target-dir
cargo-dot/config.toml	可被比赛 clone 保留的 Cargo 配置源
outputs/	汇报幻灯片等交付物
vendor/	第三方源码、离线 crates 与 BusyBox 资源

12.2 kernel/src

文件	功能
main.rs	公共入口、初始化顺序、sdcard 挂载与比赛执行器接入
console.rs	架构早期串口到 Rust 格式化输出的适配
panic.rs	panic 串口输出、单次 panic 防重入与停机
linker.rs	链接脚本符号和内核镜像物理/虚拟段描述
memory.rs	FDT 内存排除、启动页表、direct-map、buddy handoff
page_alloc.rs	全局 buddy 页分配 API 和页清零
heap.rs	Rust global allocator，连接 slab/large allocation
runtime_page_table.rs	buddy-backed 运行期页表
vm.rs	vmalloc/vfree/ioremap/ionmap 与 kernel VA 管理
fault.rs	双架构 page fault 统一分类与统计
user_mm.rs	per-process 页表、ASID、fault、copy-on-write/复制辅助
context.rs	IRQ save guard 与上下文约束
irq.rs	通用中断分类与 dispatch
irq_lock.rs	IRQ-safe spin lock
lockdep.rs	lock class/rank、持锁链和违规检测
tracked_spin.rs	带 owner/lockdep/迁移约束的自旋锁
time.rs	monotonic clock、tick 与 clockevent 策略
timer.rs	timer queue、timeout、取消和 one-shot
workqueue.rs	work、delayed work 与 worker 管理
smp.rs	CPU 生命周期、secondary bring-up、online/active mask
ipi.rs	IPI mailbox 与 reschedule/TLB/call-function 消息
call_function.rs	跨 CPU 回调请求槽和 completion
tlb.rs	kernel/per-mm TLB shutdown
trap.rs	通用 trap 初始化、解码和分发
task/mod.rs	调度器、任务状态、per-CPU run queue
task/stack.rs	64 KiB guarded kernel stack
task/wait_queue.rs	intrusive WaitQueue/Completion
task/idle_verify.rs	tickless idle/IPI 唤醒验证
task/m4c_verify.rs	抢占、迁移、wait queue 验证
task/m4c2_verify.rs	context/TLB/迁移扩展验证
process.rs	Process/Thread、fd table、signal state、child/zombie
signal.rs	signal 编号、action、mask 与发送
syscall.rs	asm-generic syscall 编号事实表
user.rs	用户入口、syscall dispatch、OSCOMP runner；当前最大集成文件
user/riscv64.S	RISC-V 用户态切换/返回汇编
user/loongarch64.S	LoongArch PLV3 切换/返回汇编
elf.rs	ELF64 解析、program header 与 relocation metadata
exec.rs	ELF 映射、动态解释器、auxv、exec image

文件	功能
initramfs.rs	newc CPIO 解析与 symlink 解析
fs/mod.rs	tmpfs/devfs/mount/path/ext4 物化集成
pipe.rs	pipe buffer、blocking、EOF、poll
tty.rs	console TTY、termios、winsize、前台进程组
devpts.rs	PTY master/slave 与 /dev/pts
procfs.rs	/proc 动态文件
sysfs.rs	/sys 设备与内核对象视图
device.rs	Bus/Device/Driver 模型
virtio.rs	VirtIO transport、DMA HAL、设备探测
block.rs	block registry、request、buffer/page cache
ext4.rs	ext4 只读 superblock/inode/directory/file
rng.rs	ChaCha20 DRBG 与 VirtIO-RNG
rtc.rs	RTC 抽象与 /dev/rtc
net/mod.rs	网络接口注册与 NetDevice trait
net/socket.rs	AF_INET TCP/UDP socket 与 syscall
net/virtio_net.rs	VirtIO-Net raw 设备包装

12.3 mm/src

文件	功能
address.rs	PhysAddr/VirtAddr 与页对齐
range.rs	半开物理范围
frame.rs	物理页帧与连续 frame block
virtual_address.rs	虚拟地址运算
virtual_page.rs	虚拟页抽象
virtual_range.rs	虚拟区间
layout.rs	虚拟布局 region 与重叠校验
map.rs	启动物理 MemoryMap 的 merge/reserve
early_allocator.rs	启动期连续页分配
paging/geometry.rs	多级页表 index 几何
paging/mapping.rs	权限、内存类型、W^X
paging/table.rs	原始 64 位页表页访问
buddy/page.rs	buddy 页 metadata/refcount
buddy/zone.rs	zone 类型与 free list
buddy/allocator.rs	order 分配/释放
slab/size_class.rs	9 个小对象 size class
slab/slab.rs	单 slab 对象与 freelist
slab/cache.rs	每 size class cache
slab/provider.rs	slab 页后端 trait
slab/allocator.rs	多 cache 分配器
heap/allocator.rs	slab + large 统一 heap
heap/large.rs	大对象连续页分配
heap/error.rs	heap 错误模型
vma.rs	VMA 集合、split、coalesce、gap
address_space.rs	用户 VMA/brk/mmap 元数据
user_space.rs	用户 fault plan、stack growth、active CPU
fault.rs	fault access/source/plan
asid.rs	ASID token、generation 与 rollover
cpu_mask.rs	原子 CPU mask
tib.rs	TLB scope/request 抽象
vmalloc.rs	kernel 虚拟区间预留与 guard
lib.rs	模块导出与 45 项单元测试入口

12.4 arch/riscv64

文件	功能
linker.ld	RISC-V 低地址 boot + 高半 kernel ELF 布局
asm/entry.S	boot CPU 两阶段启动
asm/secondary.S	secondary hart 入口
boot.rs	RISC-V 启动参数到 BootInfo
cpu.rs	WFI/FPU/CPU 原语
early_console.rs	16550 UART
interrupt.rs	SSTATUS/SIE 中断状态
sbi.rs	TIME/IPI/HSM/SRST SBI 调用
smp.rs	hart 启动与 per-CPU 状态
time.rs	time CSR 与 SBI timer
trap/entry.S	trap 保存/恢复
trap/frame.rs	RISC-V TrapFrame
trap/mod.rs	stvec/sscratch 安装与解码
task/context.rs	callee-saved Context
task/switch.S	任务上下文切换
memory/layout.rs	Sv39 用户/内核布局
memory/phys_access.rs	direct-map 物理访问
memory/paging/*	Sv39 entry/geometry/map/activate/boot

12.5 arch/loongarch64

文件	功能
linker.ld	LoongArch boot/kernel 布局
asm/entry.S	DMW 启动入口
asm/secondary.S	secondary CPU 入口
boot.rs	QEMU/EFI-style 参数转换
cpu.rs	idle、FPU 与 CPU 原语
early_console.rs	平台串口转发
interrupt.rs	CRMD/ECFG 中断状态
smp.rs	secondary CPU 与硬件 CPU ID
time.rs	CSR timer
trap/entry.S	trap 保存/恢复
trap/frame.rs	LoongArch TrapFrame
trap/mod.rs	EENTRY/SAVE CSR 安装与解码
task/context.rs	callee-saved Context
task/switch.S	上下文切换
memory/dmw.rs	cached/uncached DMW
memory/layout.rs	LA64 用户/内核布局
memory/phys_access.rs	DMW 物理访问
memory/paging/refill.S	TLB refill 汇编
memory/paging/*	四级页表 entry/geometry/map/hardware
platform/qemu_virt/boot.rs	EFI system table/FDT 定位
platform/qemu_virt/console.rs	QEMU UART
platform/qemu_virt/memory.rs	QEMU 启动保留区

12.6 基础 crate

文件	功能
boot/src/address.rs	BootAddress
boot/src/info.rs	BootInfo builder 与启动元数据
firmware/fdt/src/blob.rs	FDT header/blob 验证
firmware/fdt/src/tree.rs	CPU/memory/initrd/VirtIO/PCI 枚举
firmware/fdt/src/region.rs	MemoryRegion
firmware/fdt/src/error.rs	FDT 错误
runtime/src/console.rs	ByteConsole/ConsoleWriter
sync/src/spin_lock.rs	无依赖 SpinLock
vfs/src/lib.rs	VFS 公共对象、flags、errno、poll、dirent

12.7 scripts/

脚本按职责分组如下；.bak 文件是历史备份，不属于当前正式流程。

脚本组	代表文件	功能
提交构建	oscomp-build.sh、build.sh	生成双架构比赛内核
总审计	oscomp-audit.py	检查工具链、vendor、产物与评分入口
评分门禁	oscomp_baseline_guard.py	检查评分路径；当前有 ok 未生效缺陷
newtest 审计	oscomp-newtest-p0/p2/p3/p4/p5/p6-*.py	ABI、VFS、调度、动态 ELF、clone/futex、网络
RISC-V 启动修复审计	oscomp-riscv-*.py/.sh	high-half、lowmap、linker、allocator、stack handoff
Rust 兼容	oscomp-rust2025-*.sh/.py	edition 2024、feature gate、rust-src
sdcard	oscomp-sdcard-*.py	bounded discovery、测试执行链
preflight	oscomp-full-contest-preflight.sh	contest 前置检查
M5-M9	m5-* 至 m9*	并发、timer、用户态、用户 MM、进程 ABI 门禁
M14-M16	m14-*、m15a-*、m16*	BusyBox、ext4、动态 ELF
QEMU/smoke	run-qemu.sh、smoke.py	启动与串口 marker 验证
stress	stress-smp.py/.sh	架构/SMP/内存/profile 矩阵
BusyBox/initramfs	build-static-busybox-initramfs.*	构建静态用户态归档
ext4 工具	ext4_read.py	主机侧 ext4 读取辅助
vendor	oscomp-vendor.sh	离线依赖准备

12.8 docs/

文档	内容
boot-order.md	启动初始化依赖
context-rules.md	early/task/idle/hardirq/panic 上下文
locking.md	锁顺序与 lockdep
cpu-lifecycle.md	CPU 状态机
scheduler-state-machine.md	任务状态机
ipi-mailbox.md	IPI publish/coalesce/drain
call-function.md	跨 CPU 回调
tlb-request-v2.md	TLB request ID/target/completion
m5-completion.md	并发基础封版
m6-*.md	timer 、 workqueue 、 wait queue 与鲁棒性
m7-*.md	最小用户模式
m8*.md	用户 MM 、 demand fault 、 per-mm TLB
m9*.md	Process/Thread 与 syscall ABI
m10-completion.md	ELF/initramfs
m11-vfs.md	VFS/fd table
m12-m13-process-tty.md	clone/exec/wait/pipe/signal/TTY
m14-*.md	BusyBox 与双 vendor 用户态
m15a-ext4-ro.md	ext4 只读阶段
m16*.md	动态 ELF/auxv/preflight
oscomp_group_matrix.md	测试组分类矩阵
oscomp-submit-checklist.md	提交检查清单
ci.md	本地 release gate 与 CI 说明

13. 当前问题与风险

风险	级别	当前证据	影响
baseline guard 忽略 <code>ok</code>	P0	源码直接确认	虚假 PASS，破坏证据链
当前 HEAD 缺完整 contest 日志	P0	仅有截图和构建产物	无法逐项复现总分
LA busybox 后台 kill	P0	当前源码存在；历史 known-bad	SIGSEGV/残留子进程
glibc outer-shell fallback	P0	当前 P14K 存在；历史 0/52 等失败	busybox 组不稳定
P2 RTC 审计失败	P1	本轮可复现	VFS/device ABI 缺口
P4 动态 ELF 审计失败	P1	本轮可复现	LA loader/目录契约未统一
网络评分为 0	P1	截图	socket 源码未转化为评分
lmbench 源码与评分脱节	P1	mini runner 存在、截图 0	parser/预算/平台格式待闭环
ext4 以只读为主	P1	<code>ext4.rs</code> 设计	iozone/真实 rootfs 受限
signal 语义不完整	P1	仅基础 frame/return	libcbench/LTP 边界
<code>user.rs</code> 过大	P2	8352 行	评测、syscall、用户态强耦合
大量未跟踪备份/vendor 文件	P2	工作树状态	容易误提交或污染审查
编译 warning	P2	mm 单测 6 条	质量债务，不阻塞当前功能

14. 更新后的实施方案

阶段 A：恢复可信门禁（P0）

- 修复 `oscomp_baseline_guard.py::check()`，当 `ok=False` 时真实进入 FAIL；
- 为 PASS/FAIL/WARN/退出码增加自测；
- 删除与当前源码事实冲突的“字符串即 PASS”规则；
- 使 LA busybox 后台 kill 规则在当前 P14K 上真实失败。

完成标准：人工构造一个 false condition 时脚本退出码为 1，恢复后才为 0。

阶段 B：冻结并复现当前 741.9279 基线（P0）

- 不开启新重测试组；
- 强制重建双架构内核并记录 commit/hash；
- 当前 HEAD 各运行一次 RV/LA 完整 contest；
- 保存全部 testcase、group summary、signal、timeout 与 score；
- 把日志结果与截图矩阵逐格对齐。

完成标准：同一 HEAD 下能解释四列总分，且 summary、case 行与平台截图无矛盾。

阶段 C：稳定 LoongArch busybox（P0）

- 记录 55 case 的 primary/fallback raw；
- 移除后台 `sleep 5/kill $!` known-bad；
- 禁止不稳定的 glibc outer shell；
- 仅允许可追溯的 case-level applet fallback；
- 连续两轮 LA 无 panic、SIGSEGV、timeout。

目标：先把 53 稳定复现，再追平 RV 的 54。

阶段 D：修复 P2/P4 契约 (P1)

- 实现或明确替代 `RTC_RD_TIME` ioctl；
- 审计动态 loader 的真实 `PT_INTERP` 名称；
- 决定 `/lib64` 是真实目录还是 symlink，并让代码、审计、测试镜像一致；
- 增加 glibc/musl × RV/LA 动态 hello 与 Lua smoke。

阶段 E：提高 libcbench (P1)

优先排查四列差异：

- futex timeout/wake；
- robust list 清理；
- clone TLS/child tid；
- signal mask/delivery；
- scheduler affinity/priority；
- clock/getrusage；
- mmap/mprotect 边界。

每个修复必须先跑单 case，再跑 libcbench 组，最后跑完整 contest。

阶段 F：形成 lmbench 得分 (P1)

- 保持 RV glibc 六项最小范围；
- 确认平台需要的 normalized 格式；
- 每个值只来自真实 stdout；
- 保留 420 s 总预算、320 s 组预算与安全余量；
- 当前六项稳定后，再考虑 musl 或 LoongArch。

阶段 G：ext4 与 iotest (P2)

- 完成 ext4 write/truncate/create/unlink/rename；
- 统一 buffer cache/page cache 与 inode dirty state；
- 实现 fsync/fdatasync 持久化；
- 增加崩溃一致性和重复挂载；
- 从小文件 smoke 逐步进入 iotest。

阶段 H：网络评分 (P2)

- 确认 VirtIO-Net RX/TX queue 在两架构真实工作；
- 完成接口地址、ARP/IPv4、route 与 loopback；
- socket 阻塞/nonblock/poll/timeout；
- UDP echo → TCP connect/listen → iperf；
- 最后开启 netperf，避免同时扩大两组风险面。

阶段 I：兼容性扩展 (P3)

在所述门禁稳定后，依次推进 `cyclictst`、`libctest`、`LTP`、`allowlist`。每次只改变一个架构、一个 LibC、一个测试组，并保留可回滚提交。

15. 里程碑与完成定义

15.1 已完成或基本完成

- M0-M5：双架构内核底座、SMP/并发；
- M6：timer/timeout/workqueue；
- M7：最小用户模式；
- M8：per-process 用户 MM；
- M9：Process/Thread/syscall ABI；
- M10：ELF/initramfs；
- M11：VFS/fd table；
- M12/M13：clone/exec/wait/pipe/signal/TTY 基础；
- M14：静态 BusyBox 相邻能力；
- M15A：VirtIO block + ext4 只读；
- M16A/B：auxv 与动态 ELF 基础。

15.2 当前阶段完成定义

- [] baseline guard 能真实失败；
- [x] 两个内核产物存在且总审计识别成功；
- [x] `myos-mm` 45/45；
- [x] P0/P3/P5/P6 静态审计通过；
- [] P2/P4 静态审计通过或完成合理的契约更新；
- [] 当前 HEAD RV 完整 contest 日志；
- [] 当前 HEAD LA 完整 contest 日志；
- [] 741.9279616944203 可逐项复现；
- [] LA busybox 无 known-bad 后仍判 PASS；
- [] panic=0、scoring signal11=0、signal14=0、timeout=0；
- [] `git diff --check` 通过；
- [] 只提交预期源码、文档与交付物。

16. 维护与提交约束

- 不批量删除 `.bak`、`.oscomp_patch_backup/` 或未跟踪 vendor；它们可能是用户现场；
- 不把“有函数名”写成“运行通过”；
- 不用写死 `testcase ... success` 或 benchmark 数值替代真实执行；
- 不在一次提交中同时扩展 workload、修改 parser、改预算、改 signal/timer；
- 比赛入口 `make all` 必须保持离线、有界、无 QEMU；
- 每次 contest 保存 commit、kernel hash、完整日志、summary 和异常扫描；
- 双架构和双 LibC 的 fallback 必须记录实际执行二进制与 raw；
- 若文档、审计脚本和运行时设计冲突，先明确正确契约，再一起修改。

17. 最终判断

SudoOS-Plus 已经不是“只能启动”的实验内核：当前得分证明它能在四个架构/LibC 组合上运行基础测试、BusyBox、libcbench 和 Lua；源码则显示其已经具备双架构内存、SMP、调度、进程、VFS、动态 ELF、VirtIO 与 socket 的系统化底座。

当前瓶颈不是单纯“继续堆 syscall”，而是把三个层次重新对齐：

当前源码实现
= 当前 HEAD 可复现日志
= 平台评分截图

最优先工作应是修复失真的 baseline guard、稳定 LoongArch busybox、补齐当前 HEAD 双架构 contest 证据，再处理 P2/P4 契约。完成这些后，libcbench、lmbench、ext4 和网络才会成为可持续的增分路径，而不是一次性、难归因的试验。

附录 A：生成式人工智能使用声明

本项目在代码分析、方案整理、测试与调试辅助、文档撰写和演示文稿排版过程中使用了 Anthropic Claude 与 OpenAI Codex。生成式人工智能的输出仅作为辅助建议；所有纳入仓库的代码、脚本、测试结果和文档均由团队成员审查、修改并验证，团队对最终成果的正确性、原创性、许可证合规性和提交内容承担全部责任。生成式人工智能不作为项目作者或共同作者。第三方代码及其许可证另见附录 B。

该声明的规范文本同时用于 README.md、由 README 生成的 PDF、PPT 和 AI-使用说明.md；若后续更新，四处必须同步。

附录 B：第三方代码与依赖声明

B.1 直接 vendored 项目

组件	仓库内路径	版本/快照	上游来源	许可证	使用与修改说明
fdt	vendor/fdt-reader	0.2.0-alpha2	github.com/repnop/fdt	MPL-2.0	FDT 解析；通过 myos-fdt 封装
virtio-drivers	vendor/virtio-drivers	0.13.0	github.com/rcore-os/virtio-drivers	MIT	VirtIO transport/device；由 SudoHal 适配
smoltcp	vendor/cargo/smoltcp-0.11.0	0.11.0	github.com/smoltcp-rs/smoltcp	0BSD	TCP/UDP/IP 协议基础；当前评分网络组未闭环
vte	vendor/vte	0.15.0	github.com/alacritty/vte	Apache-2.0 OR MIT	终端解析相关依赖
lwext4	vendor/lwext4	仓库快照，未记录上游 tag	github.com/gkostka/lwext4	GPL-2.0；部分文件 BSD-3-Clause	ext4 参考/适配；许可证以各文件头和目录 LICENSE 为准
musl-cross-make	vendor/musl-cross-make	仓库快照，未记录上游 tag	github.com/richfelker/musl-cross-make	MIT	构建 musl 交叉工具链
Rust source	vendor/rust-src	nightly-2025-01-18 对应快照	github.com/rust-lang/rust	MIT OR Apache-2.0	离线 build-std；不计为团队原创代码

B.2 Cargo 离线依赖

`vendor/cargo/` 保存 `Cargo.lock` 对应的离线 `crates`。具体版本由 `Cargo.lock` 锁定，每个 `crate` 的 `.cargo-checksum.json`、`Cargo.toml` 和许可证文件保留原始归属。除为 Rust 2024/离线构建兼容所做的必要补丁外，不宣称这些依赖为团队原创。

B.3 许可证与再分发说明

- 项目首方 `crate` 在 `workspace` 中声明 `MIT OR Apache-2.0`；
- 第三方目录继续遵守各自许可证，目录内许可证优先于本项目声明；
- `lwext4` 含 GPL-2.0 文件，若其代码被链接进最终分发物，必须按 GPL-2.0 履行

源码与许可证义务；当前 Rust `kernel/src/ext4.rs` 为自研只读实现，不等同于已链接完整 `lwext4`；

- 正式提交不得删除 `vendor/` 中的版权、许可证、NOTICE 或 `checksum`；
- 新增第三方代码时，必须同步更新本附录的来源、版本、许可证和修改说明。